

---

# From a traditional to an AI-native SDLC

# AI is still new, and everyone wants it.

The tools change every month, and the pressure to adopt them comes from every direction at once. What is missing is a direction of our own.

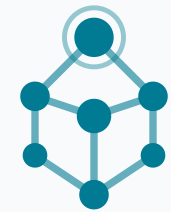
# Everyone steers, nobody drives

Every team picks its own tools, its own prompts and its own workflow. The result is fragmentation, and the direction ends up set by whoever sells the tool.

Illustration: Jan Meskens

## You are not steering. You are being driven.





# SmartAgents

---

## Training

Hands-on workshops on your own code and cases, for engineers and business teams alike.

---

## AI-native engineering

We build agents and software with agents ourselves. It is where everything else we do comes from.

---

## AI-native SDLC

We move a delivery organisation to an agentic lifecycle: quality gates, workflows and how teams work together.

---

## AI staffing

Engineers and coaches who join your teams, selected on what they have built rather than on their résumé.

# Tom Haeldermans



Founder · SmartAgents

## Career

PhD in Industrial Engineering, turned software engineer

## On holiday

Found somewhere on a mountain

## Dog dad

The walks are not up for discussion

Tom works 40% of his time for Isabel.

---

# Bas Sarrechia

Founder · SmartAgents

---

## Career

Chemist turned software engineer

---

## Obsessive

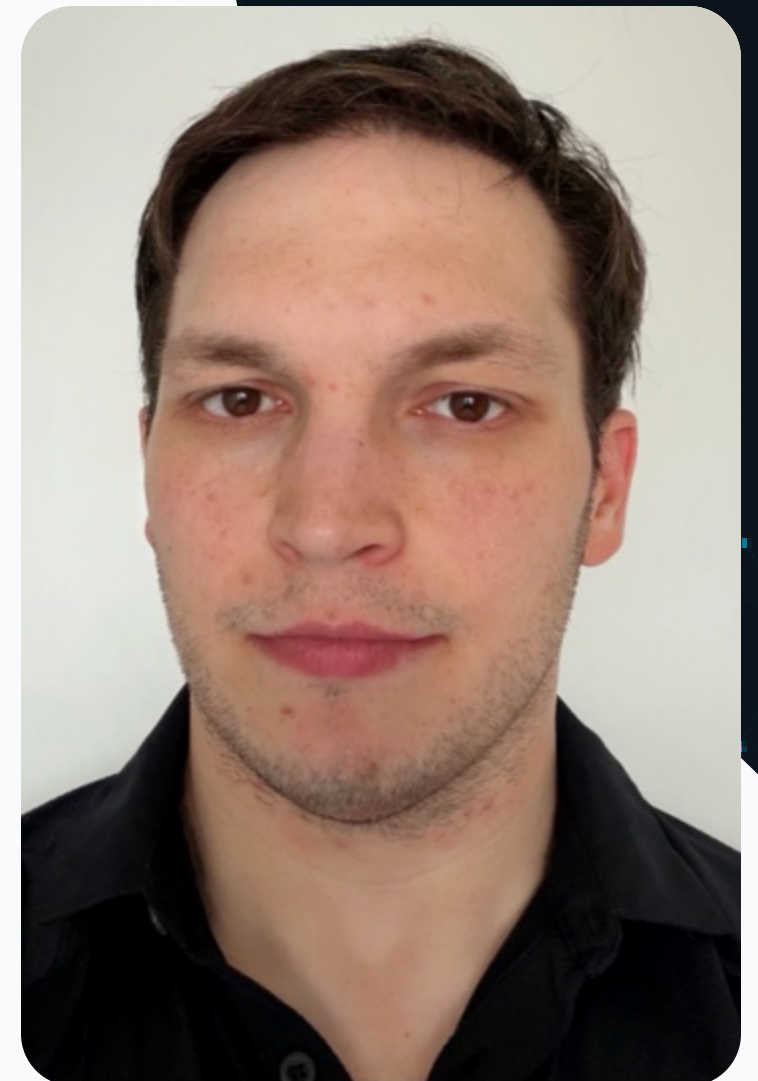
Easily, about whatever has his attention

---

## Currently

Fitness, AI and the stock market

---



Our current task: learn from within two teams

# The goal: do more with the same people

Bas Sarrechia

Ponto

Tom Haeldermans

Trustees



Then

The organisation

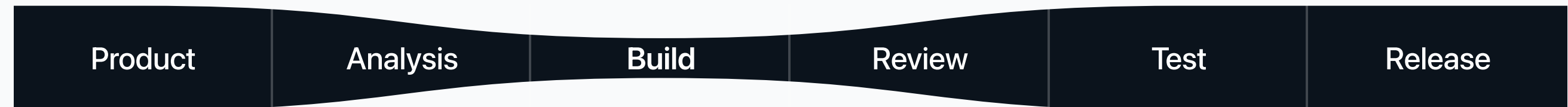
Whatever works in the two teams is put up for discussion, to roll out across the organisation.

Find out what it takes to move towards an AI-native SDLC

# The bottleneck moves. It does not disappear.

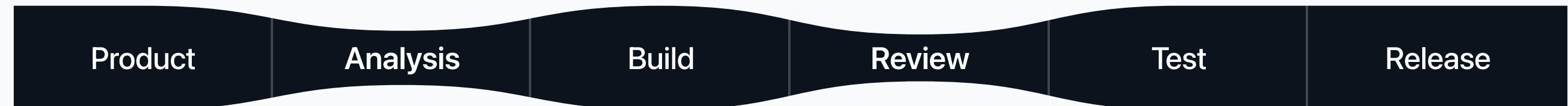
## Today

Six desks, five handoffs



## AI-assisted

What you ask and who checks it



## AI-native

One run, four decisions



Four gates 1 Intent accepted 2 Spec approved 3 Change approved 4 Release authorised

Each row narrows where work waits: Build today, then Analysis and Review, then only the release sign-off.

At each gate a person decides, and the pipeline makes sure no gate is skipped.

Adapted from Anthropic's AI-native SDLC playbook.

---

# Issues we saw in our first weeks

---

## Belief

Some distrust the output entirely, others expect miracles. It is the same mistake in opposite directions.

---

## Capability

No training, too little access to AI tooling, and processes built for a world where writing code was the expensive part.

---

## Foundation

Product knowledge lives in people's heads. AI workflows are fragmented, each only as good as whoever built it, and everything is solved with a skill.

---

## Adoption

The spread between teams is huge: some work almost AI-first, others barely use AI. There is no common way of working to close that gap.

---

---

# The hardest constraint is not technical

---

**"We've always done it this way"**

Answered by someone inside the team who shows, on real work, that it can be done differently.

---

**"I don't trust it. My code is better"**

Answered by real changes that show the output meets your standards, once the context and the gates are set up well.

---

**"It only creates slop"**

Answered by gates. Slop is what unchecked output looks like. Most teams have review and CI, but not every team applies them consistently.

---

---

# What we propose, in this order

The goal is to do more with the same people.

01

## Quality gates

Some teams already have strong gates, others far less. Every team reaches the same bar in CI/CD: today's standards, or stricter.

02

## Platform

One way of working, owned by a small platform team and carried by the tooling instead of remembered by people.

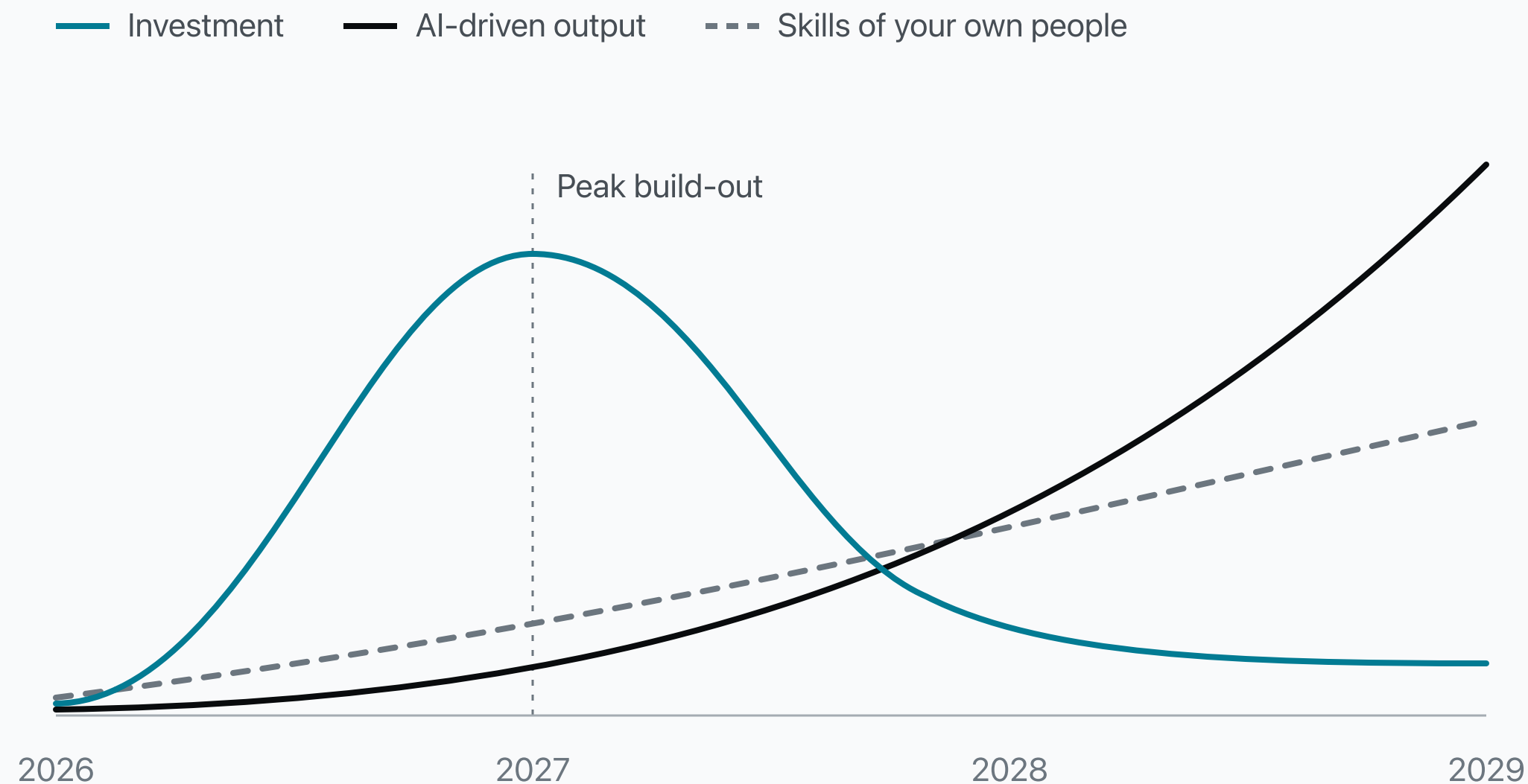
03

## Training

Teams learn to work with agents rather than around them.

Access without gates buys you more code and less confidence in it.

# We need to invest in two things



The curves are illustrative: they show the shape, not a forecast.

01 · 02

## The foundation

Quality gates that hold every change to today's standards, or stricter, and a platform that carries one way of working to every team.

03

## Your people

Training every team to work with AI, and upskilling your own people: they already hold the domain knowledge.

---

# 01 What gates check, and what that brings

## What they check

---

**Team standards:** code standards, agent guidelines and architecture rules per team.

---

**Correctness:** unit and integration tests, a coverage threshold, evals in CI, API contract and migration checks, performance budgets and accessibility checks.

---

**Security and compliance:** security, dependency and secret scans, no personal data in logs, and four-eyes approval with an audit trail.

---

**Review:** an AI reviewer checks acceptance criteria, guidelines, bugs, vulnerabilities and performance, then a person reviews the merge request.

---

## What that brings

---

**No rogue AI:** agents cannot overreach

---

**More trust in AI:** its output is checked

---

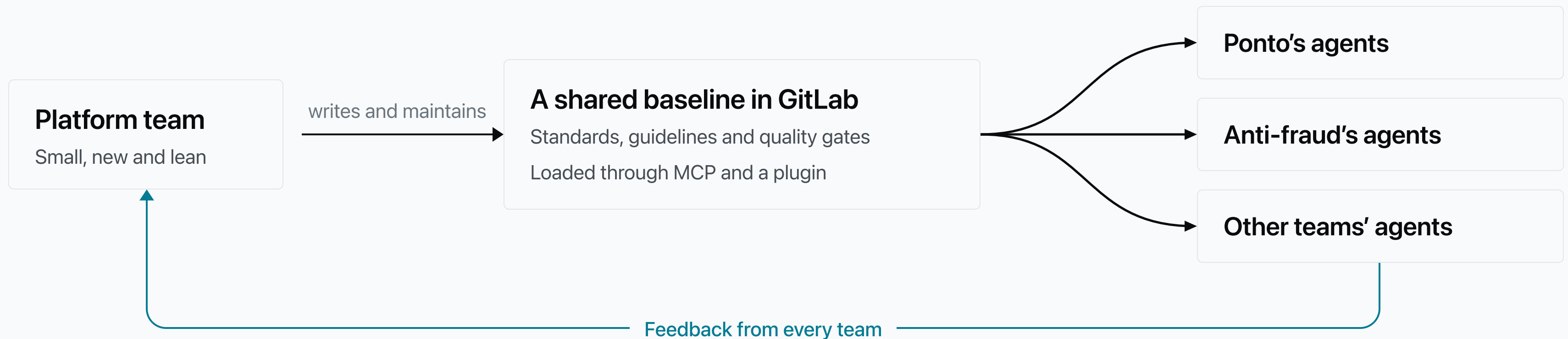
**Better output:** quality over a quick pass

---

**Standardisation:** coverage and security

---

## 02 Platform engineering



A baseline, not a limit: each team still configures and extends its own agents on top of it.

---

## 03 Where training pays off first

---

### Beyond search

AI is mostly used to look things up. The larger gain comes from handing it a whole task, end to end.

### The right model

Lighter, less expensive models do most everyday work very well. Knowing when to use which is a skill of its own.

### Proven workflows

Battle-tested workflows already exist for the common jobs. Nobody has to invent their own.

### What it costs

The cost of an AI task is rarely visible, so it is hard to steer on. Once it is, the choices get easier.

---

The willingness is there. What is missing is knowing how to use AI well, and that is what training teaches.

---

Where we are

# What we have already done.

One month inside the teams.

---

# Goal: AI-native automation of support tickets

## First results: a reusable test setup for the FICs

FICs covered by tests

16 → 99 all of them

---

Code coverage

19% → 100%

---

Along the way

---

### Readable tests

The team is happy with how the tests read.

---

### FIC configurations fixed

2 bad configurations removed, 2 others updated.

---

### 17 production issues

Identified, not yet resolved.

---

### Dead code paths

Several identified, not yet resolved.

---

# Ponto: next objectives (4 weeks)

## Follow-ups

### Clean up the findings

Resolve the production issues and dead code paths found this month.

### Logging strategy

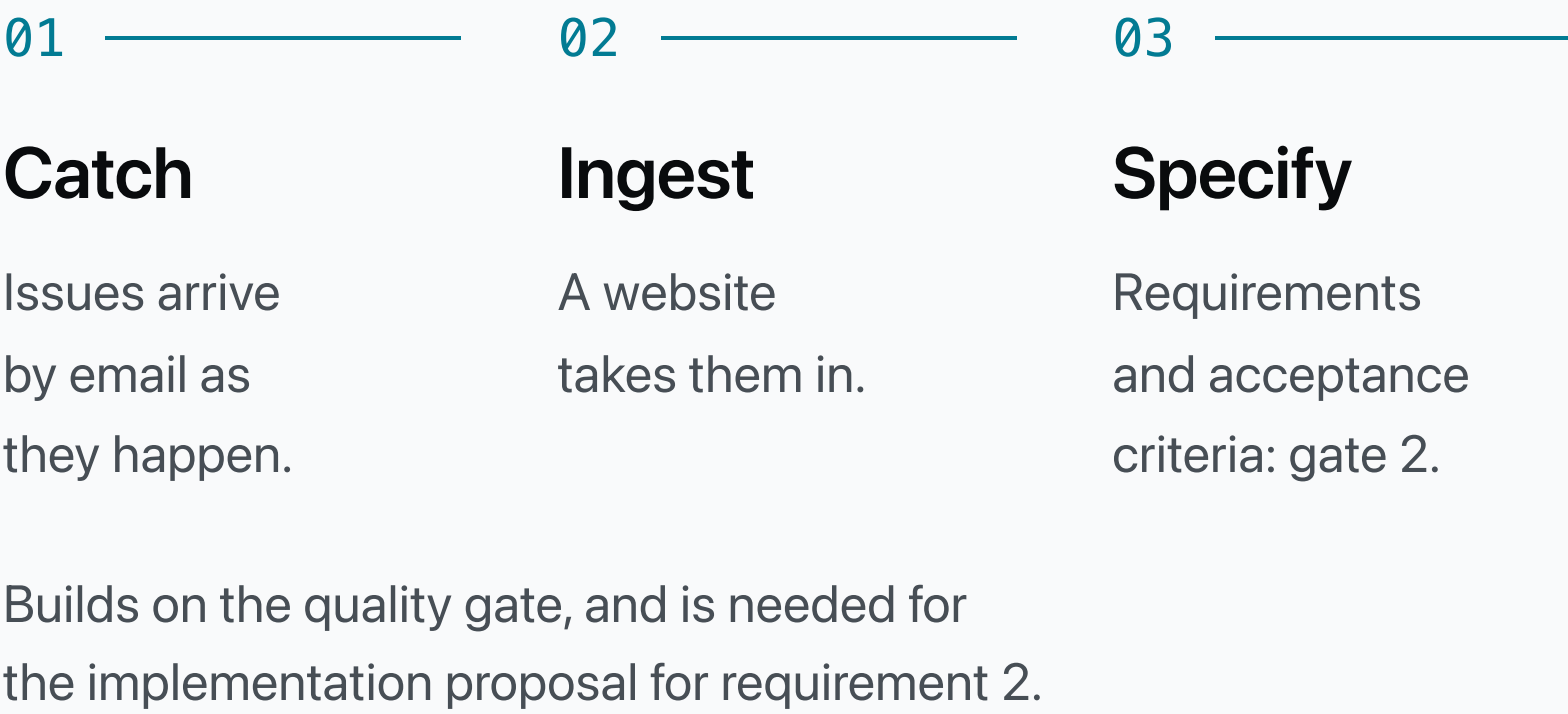
Direct impact: less load on Kibana, and it helps other issues surface along the way.

### Team coaching

Based on the team’s own /insights from Claude.

The big one

## A proposal for requirement 2



---

# Trustees: next objectives

---

## Map the way of working

Further map how the teams at TMF work with AI today, and where the skill and process gaps are.

---

## AI-native back-testing

Introduce AI-native development and ways of working on the back-testing initiative: skills, documentation and code standards, set up together with the team.

---

## A remote agent

Get a first remote AI agent running on OpenShift.

---

## Analysis handover

A structured output format and quality gates for the analysis team, before its work is handed over to development.

---

---

# Thank you

Questions or concerns?

---

[smartagents.be](https://smartagents.be)